Numpy(Numerical python) :

- used for working with array
- it is a n-dimenstion array(0D,2D,3D....)
- In python , we have list but they are slow to process so numpy amis to provide an array object that is upto 50x faster than traditional list

Numpy arrays

0-D

In [1]:
```
import numpy as np
```

In [2]:
```
a = 45
print(a)
print(type(a))
```

```
45
<class 'int'>
```

In [3]:
```
a=45
arr0 = np.array(a)
print(arr0)
type(arr0)
```

```
45
```

Out[3]:
```
numpy.ndarray
```

In [4]:
```
arr0.ndim
```

Out[4]:
```
0
```

In [5]:
```
arr0.shape  # 0D doesnt have shape, always gives empty tuple
```

Out[5]:
```
()
```

1D array

1D array represent as [ ] (list)

In [6]:
```
a =[45]
b= np.array(a)
b
```

Out[6]:
```
array([45])
```

In [7]:
```
type(b)
```

Out[7]:
```
numpy.ndarray
```

In [8]:
```
b.ndim   # ndim = len(X.shape) another way to find dim
```

Out[8]:
```
1
```

In [9]:
```
len(b.shape)
```

Out[9]:
```
1
```

shape:

(total no.of iteam ,) > this syntax(,) is only for 1D

shape always represent as ( ) tuple, whether it is 0D,1D,3D

(no of row , no of colounm) > 2D shape

In [10]:
```
b.shape
```

Out[10]:
```
(1,)
```

In [11]:
```
a= [1,2,3,4,5]
b= np.array(a)
print(b)
print(type(b))
print(b.ndim)
print(b.shape)
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
1
(5,)
```

funtions for creating 1D aaray

arange :

1. np.arange(start,end,step) end:exclude

2.range(core python)=arange(numpy) both are same

In [12]:
```
a= np.arange(10)
a
```

Out[12]:
```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [13]:
```
a=np.arange(1,10)
a
```

Out[13]:
```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

In [14]:
```
a=np.arange(1,12,2)
a
```

Out[14]:
```
array([ 1,  3,  5,  7,  9, 11])
```

2.linespace

syntax: np.linespace(start , end ,no of iteam) , end is included

b/w two iteam distance is equal

In [15]:
```
a=np.linspace(10,20,5)
a
```

Out[15]:
```
array([10. , 12.5, 15. , 17.5, 20. ])
```

In [16]:
```
print(12.5-10)
print(15-12.5)
```

```
2.5
2.5
```

3.zeros

syntax = np.zeros((shape))

In [17]:
```
a= np.zeros(4,)
a
```

Out[17]:
```
array([0., 0., 0., 0.])
```

In [18]:
```
a= np.zeros((4,4),dtype=int)
a
```

Out[18]:
```
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

In [19]:
```
a+2
```

Out[19]:
```
array([[2, 2, 2, 2],
       [2, 2, 2, 2],
       [2, 2, 2, 2],
       [2, 2, 2, 2]])
```

In [20]:
```
a*1
```

Out[20]:
```
array([[0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0],
       [0, 0, 0, 0]])
```

ones

syntax : np.ones((shape))

In [21]:
```
a=np.ones(3,)
a
```

Out[21]:
```
array([1., 1., 1.])
```

In [22]:
```
a=np.ones((3,4),dtype=int)
a
```

Out[22]:
```
array([[1, 1, 1, 1],
       [1, 1, 1, 1],
       [1, 1, 1, 1]])
```

array funtion

In [23]:
```
a=np.random.random((3,))  #return random 3 value as float b/w 0 to 1
a
```

Out[23]:
```
array([0.0098475, 0.9795372, 0.4851831])
```

a=np.random.randint(strt,end,no of iteam)

strt(include) and end(exclude)

In [24]:
```
a=np.random.randint (10,20,5)
a
```

Out[24]:
```
array([10, 16, 14, 12, 19], dtype=int32)
```

indexing and slicing :

array = list(all indexing and slicing rule of list is apllicable for array)

In [28]:
```
a=np.arange(10,51,10)
print(a)
```

```
[10 20 30 40 50]
```

accessing single value:

In [31]:
```
print(a[-1])
```

```
50
```

In [32]:
```
print(a[0])
```

```
10
```

accessing multiple value:

In [38]:
```
print(a)
```

```
[10 20 30 40 50]
```

In [40]:
```
a[[0,1,4]]
```

Out[40]:
```
array([10, 20, 50])
```

accessing multiple value through slicing:

In [34]:
```
print(a)
```

```
[10 20 30 40 50]
```

In [35]:
```
a[0:5]
```

Out[35]:
```
array([10, 20, 30, 40, 50])
```

In [36]:
```
a[1:5:1]
```

Out[36]:
```
array([20, 30, 40, 50])
```

In [37]:
```
a[:-1]
```

Out[37]:
```
array([10, 20, 30, 40])
```

replacing single,multiple value through indexing

In [41]:
```
print(a)
```

```
[10 20 30 40 50]
```

In [44]:
```
a[0]=100
print(a)
```

```
[100  20  30  40  50]
```

In [47]:
```
a[[1,3]]=200,300
print(a)
```

```
[100 200  30 300  50]
```

modify the numpy value

syntax= np.append(array,value)

In [52]:
```
a=np.arange(1,11)
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

In [49]:
```
a=np.append(a,11)
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11]
```

In [50]:
```
a=np.append(a,[12,13,14])
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14]
```

delete a single and multiple value in numpy

In [53]:
```
print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

In [54]:
```
b = np.delete(a,-1)
print(b)
```

```
[1 2 3 4 5 6 7 8 9]
```

In [55]:
```
b = np.delete(a,[0,1,3])
print(b)
```

```
[ 3  5  6  7  8  9 10]
```

maths operation and comparison :

In [56]:
```
a = np.arange(1,6)
b=np.arange(10,16)
c=np.arange(2,7)
print(a)
print(b)
print(c)
```

```
[1 2 3 4 5]
[10 11 12 13 14 15]
[2 3 4 5 6]
```

In [58]:
```
print(a+c)
```

```
[ 3  5  7  9 11]
```

In [59]:
```
print(a-c)
```

```
[-1 -1 -1 -1 -1]
```

In [60]:
```
print(a*c)
```

```
[ 2  6 12 20 30]
```

In [61]:
```
print(b**2)
```

```
[100 121 144 169 196 225]
```

comparison

In [62]:
```
a==c
```

Out[62]:
```
array([False, False, False, False, False])
```

In [64]:
```
a<c
```

Out[64]:
```
array([ True,  True,  True,  True,  True])
```

In [66]:
```
np.array_equal(a,c) # all iteam should be equal otherwise it will throw false
```

Out[66]:
```
False
```

In [67]:
```
print(a)
```

```
[1 2 3 4 5]
```

In [68]:
```
np.exp(a)
```

Out[68]:
```
array([  2.71828183,   7.3890561 ,  20.08553692,  54.59815003,
       148.4131591 ])
```

In [69]:
```
np.log(a)
```

Out[69]:
```
array([0.        , 0.69314718, 1.09861229, 1.38629436, 1.60943791])
```

In [70]:
```
np.log(0)
```

```
C:\Users\User\AppData\Local\Temp\ipykernel_9028\2933082444.py:1: RuntimeWarning:
divide by zero encountered in log
  np.log(0)
```

Out[70]:
```
np.float64(-inf)
```

In [71]:
```
import warnings
warnings.simplefilter('ignore')
```

In [72]:
```
np.log(0)
```

Out[72]:
```
np.float64(-inf)
```

1D array completed